

ABCDryBASIN

Formula & Calculation Reference Manual

Drought Risk Assessment Platform

Comprehensive documentation of all mathematical models,
algorithms, and data processing methods

Generated: June 2026

Project: ABCDryBASIN

Table of Contents

1. Soil Physics - Saxton-Rawls Pedotransfer Functions	3
2. Evapotranspiration - FAO-56 Penman-Monteith	4
3. Crop Coefficient (Kc) Model	5
4. Soil Water Balance	6
5. Drought Indices (SPI, VCI)	7
6. Composite Risk Scoring	8
7. Heuristic Driver Scores & Ensemble	9
8. XGBoost Machine Learning Model	10
9. LSTM Neural Network Forecasting	11
10. Rolling Window Aggregations	12
11. Haversine Coordinate Validation	13
12. Synthetic Data Generation	14
13. Classification Thresholds	15
14. JavaScript / Turf.js Spatial Calculations	16
15. Data Validation & Unit Conversion	17
16. Statistical Evaluation Metrics	18

1. Soil Physics - Saxton-Rawls Pedotransfer Functions

These empirical functions estimate soil hydraulic properties from texture and organic matter. Based on Saxton & Rawls (2006). All percentages are 0-100 scale.

$$\begin{aligned} FC = & -0.251*SAND + 0.195*CLAY + 0.011*OM + 0.006*(SAND*OM) \\ & - 0.027*(CLAY*OM) + 0.452*SILT + 0.299 \end{aligned}$$

Description: Field Capacity - water content at field capacity (%)

Inputs: SAND%, CLAY%, SILT%, OM% (organic matter)

Source: Saxton & Rawls (2006) pedotransfer function

Reference: *prediction_engine/physical_helpers.py:4*

$$\begin{aligned} WP = & -0.024*SAND + 0.487*CLAY + 0.006*OM + 0.005*(SAND*OM) \\ & - 0.013*(CLAY*OM) + 0.068*SILT + 0.031 \end{aligned}$$

Description: Wilting Point - water content at permanent wilting point (%)

Inputs: SAND%, CLAY%, SILT%, OM% (organic matter)

Source: Saxton & Rawls (2006) pedotransfer function

Reference: *prediction_engine/physical_helpers.py:9*

$$AWC_mm = (FC\% - WP\%) / 100 * ROOT_DEPTH_mm$$

Description: Available Water Capacity - total plant-available water in root zone (mm)

Inputs: FC% (field capacity), WP% (wilting point), ROOT_DEPTH_mm (root zone depth)

Source: Derived from Saxton-Rawls outputs

Reference: *prediction_engine/physical_helpers.py:14*

2. Evapotranspiration - FAO-56 Penman-Monteith

The FAO-56 Penman-Monteith equation is the standard method for computing reference evapotranspiration (ET₀) from meteorological data.

$$ET_0 = \frac{[0.408 \cdot D \cdot (R_n - G) + g \cdot (900 / (T + 273)) \cdot u_2 \cdot (e_s - e_a)]}{[D + g \cdot (1 + 0.34 \cdot u_2)]}$$

Description: Reference Evapotranspiration (mm/day)

Inputs: R_n = net radiation (MJ/m²/day), G = soil heat flux (~0), T = mean temp (C), u₂ = wind speed at 2m (m/s), e_s-e_a = vapor pressure deficit

Source: FAO Irrigation & Drainage Paper 56

Reference: *prediction_engine/physical_helpers.py:19*

Supporting Calculations

$$D = 4098 \cdot 0.6108 \cdot \exp((17.27 \cdot T) / (T + 237.3)) / (T + 237.3)^2$$

Description: Slope of saturation vapor pressure curve (kPa/C)

Inputs: T = mean daily temperature (C)

Source: FAO-56

Reference: *physical_helpers.py:22*

$$e_s = 0.6108 \cdot \exp((17.27 \cdot T) / (T + 237.3))$$

Description: Saturation vapor pressure (kPa)

Inputs: T = mean daily temperature (C)

Source: FAO-56

Reference: *physical_helpers.py:24*

$$e_a = e_s \cdot \text{HUMIDITY\%} / 100$$

Description: Actual vapor pressure (kPa)

Inputs: e_s (saturation vapor pressure), HUMIDITY% (relative humidity)

Source: FAO-56

Reference: *physical_helpers.py:26*

$$R_n = \text{SOLAR_RAD_Wm2} \cdot 0.0864 \cdot (1 - 0.23)$$

Description: Net radiation (MJ/m²/day). Albedo = 0.23 (reference grass).

Inputs: SOLAR_RAD_Wm2 (solar radiation in W/m²)

Source: FAO-56

Reference: *physical_helpers.py:30*

$$g = 0.665e-3 * 101.3$$

Description: Psychrometric constant (kPa/C) at sea level

Inputs: Atmospheric pressure (101.3 kPa at sea level)

Source: FAO-56

Reference: *physical_helpers.py*:28

$$ET_c = ET_0 * K_c$$

Description: Crop Evapotranspiration (mm/day)

Inputs: ET_0 (reference ET), K_c (crop coefficient)

Source: FAO-56

Reference: *prediction_engine/physical_helpers.py*:49

3. Crop Coefficient (Kc) Model

The crop coefficient K_c is determined by a piecewise linear function over four growth stages (initial, development, mid-season, late-season). Based on FAO-56.

Stage Definition & Kc Values

Crop	Initial	Develop	Mid	Late	End
Wheat / Barley	0.30	Linear	1.15	Linear	0.30
Olive	0.65	Linear	0.70	Linear	0.70
Date Palm	0.65	Linear	0.95	Linear	0.50
Alfalfa	0.65	Linear	1.20	Linear	0.30

During development and late-season stages, K_c varies linearly between the adjacent stage values. K_c is constrained to a maximum of 2.0 for physical plausibility.

```
Kc_total = interpolate(stage_duration_fraction, Kc_start, Kc_end)
```

Description: Linear interpolation between stage boundaries based on days since planting

Inputs: days_since_planting, total_growth_days, crop_type

Source: FAO-56 approach

Reference: *prediction_engine/physical_helpers.py*:32

4. Soil Water Balance

Daily soil water accounting based on inputs and losses. The soil profile is treated as a single-layer reservoir bounded by 0 and AWC.

$$SM_t = SM_{(t-1)} + RAIN + IRRIGATION - ETc - RUNOFF$$

Description: Daily soil moisture update (mm). Constrained between 0 and AWC.

Inputs: $SM_{(t-1)}$ = previous day moisture (mm), RAIN = daily rainfall (mm), IRRIGATION = applied water (mm), ETc = crop ET (mm), RUNOFF = runoff (mm)

Source: Single-layer bucket model

Reference: *prediction_engine/physical_helpers.py:54*

5. Drought Indices

Standardized Precipitation Index (SPI)

$$SPI = (RAIN_{30d_SUM} - HIST_MEAN) / HIST_STD$$

Description: Standardized measure of rainfall deviation. Positive = wet, negative = dry.

Inputs: RAIN_30d_SUM = sum of rainfall over last 30 days, HIST_MEAN = historical mean of 30-day sums, HIST_STD = historical standard deviation

Source: McKee et al. (1993)

Reference: *prediction_engine/physical_helpers.py:60*

Vegetation Condition Index (VCI)

$$VCI = (NDVI_{current} - NDVI_{min}) / (NDVI_{max} - NDVI_{min}) * 100$$

Description: Vegetation health relative to historical range (0-100%). Low values indicate vegetation stress.

Inputs: NDVI_current (current NDVI value), NDVI_min/NDVI_max (historical min/max NDVI)

Source: Kogan (1995)

Reference: *prediction_engine/physical_helpers.py:68*

6. Composite Risk Scoring

Three different risk score formulations exist, used in different contexts:

A. Dashboard View Risk Score (views.py)

```
SCORE = (w_NDVI*NDVI_score + w_T*T_score + w_R*RAIN_score + w_H*HUM_score)
        / TOTAL_WEIGHT
```

Description: Weighted average of normalized metric scores. Each metric normalized 0-100.

Inputs: NDVI, temperature, rainfall, humidity observed values. Normalization: $(val - min)/(max - min) * 100$

Source: Custom heuristic

Reference: *dashboard/views.py:78*

B. XGBoost Target Risk Score (xgboost_risk_predictor.py)

```
SM_risk = max(0, min(100, (1 - SM/FC)*100))
NDVI_risk = max(0, min(100, (1 - NDVI)*100))
RAIN_risk = max(0, min(100, max(0, 100 - RAIN_30d*2)))
T_risk = max(0, min(100, (Tmax - 35)*5)) if Tmax>35 else 0

FINAL = SM_risk*0.4 + NDVI_risk*0.25 + RAIN_risk*0.2 + T_risk*0.15
```

Description: Physical-based composite risk used as training target for XGBoost model (0-100)

Inputs: SM = soil moisture, FC = field capacity, NDVI = vegetation index, RAIN_30d = 30-day rainfall sum, Tmax = max temperature (C)

Source: Custom physical composite

Reference: *prediction_engine/xgboost_risk_predictor.py:67*

C. Risk Level Classification

```
< 20  -> "Normal"
20-39 -> "Watch"
40-59 -> "Moderate"
60-79 -> "Severe"
>=80  -> "Extreme"
```

Description: Risk level labels derived from the 0-100 continuous score

Inputs: risk_score (float 0-100)

Source: Custom thresholds

Reference: *All risk components (views.py:86, llm_explainer.py:26)*

7. Heuristic Driver Scores & Ensemble Prediction

The pipeline.py orchestrates physical heuristic calculations and combines them with ML model outputs.

Driver Scores (each 0-100)

```
MOISTURE_driver = max(0, min(100, (1 - avg_SM/FC)*100))
RAINFALL_driver = max(0, min(100, max(0, 100 - RAIN_30d*2)))
TEMP_driver = max(0, min(100, (Tmax-35)*10)) if Tmax>35 else 0
NDVI_driver = max(0, min(100, (1 - avg_NDVI)*100))
HUMIDITY_driver = max(0, min(100, (100 - avg_HUM)*1.5))
```

Description: Individual risk drivers computed from observed/simulated data

Inputs: SM, FC, RAIN_30d, Tmax, NDVI, HUM from the prediction frame

Source: Custom heuristic

Reference: *prediction_engine/pipeline.py:356*

Heuristic Risk

```
HEUR_RISK = MOISTURE*0.35 + RAINFALL*0.20 + TEMP*0.15 + NDVI*0.20 + HUMIDITY*0.10
```

Description: Weighted combination of driver scores into a single heuristic risk estimate

Inputs: 5 driver scores (0-100 each)

Source: Custom weighting

Reference: *prediction_engine/pipeline.py:375*

Ensemble Prediction

```
FINAL_RISK = (HEUR_RISK + XGB_RISK) / 2
```

Description: Simple average of heuristic and ML model outputs when XGBoost is available

Inputs: heuristic_risk, xgboost_risk (both 0-100)

Source: Custom ensemble

Reference: *prediction_engine/pipeline.py:444*

Derived Columns in Pipeline

```
* etc_mm = crop_coefficient_kc * et0_mm
* soil_moisture_mm = moisture_pct / 100 * root_depth_mm
* awc_mm = (fc - wp) / 100 * root_depth_mm
* water_deficit_mm = max(0, awc_mm - soil_moisture_mm)
* deficit_pct = water_deficit_mm / awc_mm * 100
* daily_water_balance = rainfall - etc
* cumulative_water_balance = cumsum(water_balance) over lookback
```

8. XGBoost Machine Learning Model

Gradient boosted tree regressor trained on engineered features from a 30-day sliding window. Uses physical composite risk as the target variable.

Feature Engineering (30+ features)

- * soil_moisture_mean/std/min/max/trend (30-day rolling)
- * rainfall_sum_30day, rainfall_days_with_rain, rainfall_intensity
- * temp_max_mean, temp_max_trend, temp_min_mean
- * humidity_mean, humidity_trend
- * et0_sum (30-day ET sum)
- * ndvi_mean, ndvi_trend
- * awc_mm, field_capacity_pct (contextual)

Model Configuration

```
XGBRegressor(
    n_estimators=300, max_depth=6, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8, random_state=42
)
```

Description: XGBoost model with regularization via subsampling and column sampling

Inputs: Feature vector (~30 features), Target = risk_score (0-100)

Source: XGBoost library (Chen & Guestrin, 2016)

Reference: [prediction_engine/xgboost_risk_predictor.py:85](#)

Preprocessing

$$Z = (X - \text{MEAN}) / \text{STD}$$

Description: StandardScaler normalization applied per feature column

Inputs: Training features

Source: scikit-learn StandardScaler

Reference: [prediction_engine/xgboost_risk_predictor.py:110](#)

Driver Extraction (Feature Importance)

$$\text{DRIVER}_i = \text{IMPORTANCE}_i / \text{SUM}(\text{IMPORTANCE}) * 100$$

Description: Top-5 most important features normalized to 100% for drought driver attribution

Inputs: XGBoost model.feature_importances_

Source: Custom normalization

Reference: [prediction_engine/xgboost_risk_predictor.py:150](#)

9. LSTM Neural Network Forecasting

A recurrent neural network for multi-step time series forecasting of soil moisture. Uses a 30-day lookback to predict 7-30 days ahead.

Model Architecture

```
Input(30 days x 10 features)
-> LSTM(hidden=64, layers=2, dropout=0.2)
  -> Linear(64 -> 1)
    -> Output: next day soil moisture
```

Description: 2-layer LSTM with dropout regularization, followed by a linear output layer

Inputs: 10 features: soil_moisture%, rainfall, Tmax, Tmin, humidity%, wind_speed, solar_radiation, NDVI, LST, ETC

Source: PyTorch nn.Module

Reference: *prediction_engine/lstm_forecaster.py:16*

Sequence Preparation

```
X[i] = sequence[i : i+SEQ_LEN]
y[i] = target[i+SEQ_LEN-1+FORECAST_DAYS]
```

Description: Sliding window: each sample is a SEQ_LEN-length window, target is FORECAST_DAYS ahead of the window end

Inputs: Full time series dataframe, SEQ_LEN=30, FORECAST_DAYS=7

Source: Custom sliding window

Reference: *prediction_engine/lstm_forecaster.py:70*

Normalization

```
X_norm = (X - MEAN) / (STD + 1e-8)
```

Description: Per-column z-score normalization. Epsilon prevents division by zero.

Inputs: Raw numeric feature columns

Source: StandardScaler analogue

Reference: *prediction_engine/lstm_forecaster.py:52*

Training Configuration

```
LOSS = MSE(y_true, y_pred)
OPTIMIZER = Adam(lr=1e-3)
EARLY_STOP = patience=8 on validation loss
```

Description: Mean squared error loss with Adam optimizer. Early stopping prevents overfitting.

Inputs: y_true (target risk), y_pred (model output)

Source: PyTorch MSELoss, Adam optimizer

Reference: *prediction_engine/lstm_forecaster.py:93*

Recursive Multi-Step Forecast

```
For day = 1..FORECAST_DAYS:  
    pred = model(last_30_days)  
    sequence = sequence[1:] + [pred]  
    forecasts.append(pred)
```

Description: Recursive: each prediction feeds back into the input window for the next step

Inputs: Last 30 days of observed data

Source: Custom recursive forecasting

Reference: *prediction_engine/lstm_forecaster.py:173*

Risk from LSTM Forecast

```
RISK = max(0, min(100, (1 - SM_predicted/FC)*100))
```

Description: Converts forecasted soil moisture back into a drought risk score (0-100)

Inputs: SM_predicted (mm), FC = field capacity (mm)

Source: Custom

Reference: *prediction_engine/lstm_forecaster.py:198*

10. Rolling Window Aggregations

Daily data is resampled and aggregated over sliding windows to capture recent trends and cumulative effects.

Metric	Formula
7d rainfall sum	SUM(rainfall) over 7-day window
30d rainfall sum	SUM(rainfall) over 30-day window
7d ET sum	SUM(ETc) over 7-day window
30d ET sum	SUM(ETc) over 30-day window
7d avg temperature	MEAN(Tmean) over 7-day window
30d avg temperature	MEAN(Tmean) over 30-day window
7d avg NDVI	MEAN(NDVI) over 7-day window
30d avg NDVI	MEAN(NDVI) over 30-day window
Water balance	RAIN - ETc (daily)
Cumulative water balance	cumsum(water_balance) over lookback window

Source: prediction_engine/pipeline.py:219-260

11. Haversine Coordinate Validation

Validates that submitted coordinates fall within an acceptable distance from the region's defined center point. Used during data ingestion to enforce geographic bounds.

```
a = sin^2(Dlat/2) + cos(lat1)*cos(lat2)*sin^2(Dlon/2)
c = 2 * atan2(sqrt(a), sqrt(1-a))
d = 6371 * c
```

Description: Great-circle distance between two points on Earth (km)

Inputs: lat1, lon1 = region center point (degrees), lat2, lon2 = submitted coordinates (degrees)

Source: Haversine formula

Reference: *dashboard/views.py:157*

```
VALID if d <= REGION.radius_km
REGIONS: Countries = 300 km, Sub-regions = 50 km
Defaults to 100 km for unspecified regions
```

Description: Validation rule: submitted coordinates must be within region's radius

Inputs: d = haversine distance (km), region.radius_km = configured max radius

Source: Custom geographic bounds check

Reference: *dashboard/views.py:165*

12. Synthetic Data Generation

Synthetic datasets are generated using stochastic processes with seasonal patterns for testing and model training.

Rainfall (Gamma Distribution)

```
RAIN = Gamma(shape=2.0, scale=5.0) per day
```

Description: Daily rainfall (mm), non-negative

Inputs: Random seed

Source: Gamma distribution

Reference: `scripts/generate_sample_drought_data.py`

Temperature (Seasonal + Noise)

```
Tmax = 35 + 15*sin(2p*day/365 - p/2) + N(0,3)
Tmin = 18 + 10*sin(2p*day/365 - p/2) + N(0,3)
```

Description: Seasonal temperature with Gaussian noise

Inputs: day (1-365)

Source: Custom seasonal model

Reference: `scripts/generate_sample_drought_data.py`

Relative Humidity (Seasonal + Noise)

```
HUM = 55 + 20*sin(2p*day/365 + p/2) + N(0,8)
```

Description: Seasonal relative humidity with noise, clipped to 0-100

Inputs: day (1-365)

Source: Custom seasonal model

Reference: `scripts/generate_sample_drought_data.py`

NDVI (Logistic Growth + Stress)

```
NDVI = 0.15 + 0.60*logistic(days_planting)*exp(-0.01*rain_deficit)
```

Description: Crop growth curve with drought stress decay

Inputs: days_since_planting, rainfall_deficit

Source: Custom crop growth model

Reference: `scripts/generate_sample_drought_data.py`

NDWI (Water Index)

$$\text{NDWI} = -0.1 + 0.5 * (\text{SM} / \text{FC}) - 0.1 * \text{NDVI} + N(0, 0.05)$$

Description: Water index derived from soil moisture and vegetation

Inputs: SM = soil moisture, FC = field capacity, NDVI

Source: Custom

Reference: *scripts/generate_sample_drought_data.py*

PDSI Approximation (Simplified)

$$\text{PDSI} = -3.0 * (1 - \text{RAIN}_{30\text{d}} / \text{ET0}_{30\text{d}}) + N(0, 0.5)$$

Description: Simplified Palmer Drought Severity Index approximation

Inputs: RAIN_30d, ET0_30d (30-day totals)

Source: Simplified Palmer model

Reference: *scripts/generate_sample_drought_data.py*

Synthetic Target Risk Score

$$\begin{aligned} \text{RISK} = & 0.30 * (100 - \text{NDVI} * 100) + 0.25 * \text{SM_risk} + 0.20 * \text{RAIN_risk} \\ & + 0.15 * \text{TEMP_risk} + 0.10 * \text{HUM_risk} \end{aligned}$$

Description: Composite target risk score for ML model training

Inputs: NDVI, soil_moisture, rainfall_30d, Tmax, humidity

Source: Custom weighted composite

Reference: *scripts/generate_sample_drought_data.py*

13. Classification Thresholds

Continuous values are classified into discrete categories using fixed thresholds.

SPI Classification

SPI Range	Classification
< -2.0	Extremely Dry
-2.0 to -1.5	Severely Dry
-1.5 to -1.0	Moderately Dry
-1.0 to 1.0	Near Normal
1.0 to 1.5	Moderately Wet
1.5 to 2.0	Very Wet
> 2.0	Extremely Wet

NDVI Classification

NDVI Range	Classification
< 0.1	Barren
0.1 - 0.2	Sparse
0.2 - 0.4	Moderate
> 0.4	Dense Vegetation

VCI Classification

VCI Range	Classification
< 10	Extreme Drought
10 - 25	Severe Drought
25 - 40	Moderate Drought
40 - 60	Normal
> 60	Very Good

Source: [Templates/dashboard/analysis.html](#)

14. JavaScript / Turf.js Spatial Calculations

Client-side spatial operations performed in the browser using the Turf.js library.

```
distance = turf.distance(point1, point2, {units:"kilometers"})
```

Description: Great-circle distance between two map points (km)

Inputs: Two GeoJSON Point objects

Source: Turf.js (Haversine)

Reference: [static/dashboard/js/dashboard.js](#)

```
area_hectares = turf.area(polygon) / 10000
```

Description: Area of a drawn polygon converted from m2 to hectares

Inputs: GeoJSON Polygon object

Source: Turf.js

Reference: [static/dashboard/js/dashboard.js](#)

```
bearing = turf.bearing(from, to)
```

Description: Compass bearing between two points (degrees from north)

Inputs: Two GeoJSON Point objects

Source: Turf.js

Reference: [static/dashboard/js/dashboard.js](#)

```
delta_pct = (current - previous) / previous * 100
```

Description: Percent change for real-time dashboard updates (wind, rain)

Inputs: current, previous sensor values

Source: Custom

Reference: [static/dashboard/js/dashboard.js](#)

15. Data Validation & Unit Conversion

Ingested data is validated against agronomically realistic bounds and converted to standard units.

Validation Bounds

Metric Type	Valid Range
NDVI / NDWI	-1.0 to 1.0
Moisture, Humidity, OM (%)	0 to 100
Temperature (C)	-50 to 60
Evapotranspiration (mm/day)	0 to 20
Salinity	0 to 50
SPI	-5 to 5
Wind Speed	0 to 200
Rainfall / Irrigation / Runoff	>= 0

Wind Speed Conversion

```
km/h -> m/s: val / 3.6
m/s -> km/h: val * 3.6
```

Description: Unit normalization for ET calculations

Inputs: Wind speed in original units

Source: Standard unit conversion

Reference: `dashboard/data_ingestion_utils.py:157`

StandardScaler Normalization (ML)

```
Z = (X - MEAN) / STD
(with epsilon 1e-8 to prevent division by zero)
```

Description: Standard score normalization for ML model inputs

Inputs: Raw feature matrix X

Source: scikit-learn StandardScaler / custom

Reference: `xgboost_risk_predictor.py`, `lstm_forecaster.py`

Metric Score Normalization (views.py)

```
NORM = (VALUE - MIN_BOUND) / (MAX_BOUND - MIN_BOUND) * 100
```

Description: Normalizes any metric to 0-100 scale for risk scoring

Inputs: observed value, min_bound, max_bound

Source: Min-max normalization

Reference: `dashboard/views.py`

16. Statistical Evaluation Metrics

Model performance is evaluated using standard regression metrics.

$$\text{RMSE} = \sqrt{\text{MEAN}((y_{\text{true}} - y_{\text{pred}})^2)}$$

Description: Root Mean Squared Error - penalizes large errors more heavily

Inputs: y_{true} (actual values), y_{pred} (predicted values)

Source: Standard regression metric

Reference: *train_xgboost_quick.py*:65, *train_lstm_quick.py*:157

$$\text{MAE} = \text{MEAN}(|y_{\text{true}} - y_{\text{pred}}|)$$

Description: Mean Absolute Error - average absolute deviation

Inputs: y_{true} , y_{pred}

Source: Standard regression metric

Reference: *train_xgboost_quick.py*:66, *train_lstm_quick.py*:158

$$R^2 = 1 - \frac{\text{SUM}((y_{\text{true}} - y_{\text{pred}})^2)}{\text{SUM}((y_{\text{true}} - \text{MEAN}(y_{\text{true}}))^2)}$$

Description: Coefficient of determination - proportion of variance explained (0 to 1)

Inputs: y_{true} , y_{pred}

Source: Standard regression metric

Reference: *train_xgboost_quick.py*:67

$$\text{MSE} = \text{MEAN}((y_{\text{true}} - y_{\text{pred}})^2)$$

Description: Mean Squared Error - training loss for both XGBoost and LSTM

Inputs: y_{true} , y_{pred}

Source: Standard loss function

Reference: *train_xgboost_quick.py*, *lstm_forecaster.py*

End of Reference Manual

This document covers all mathematical formulas and calculation methods used in the ABCDryBASIN drought risk assessment platform.

For the latest version, refer to the [source code repository](#).